

Sprint QA Process

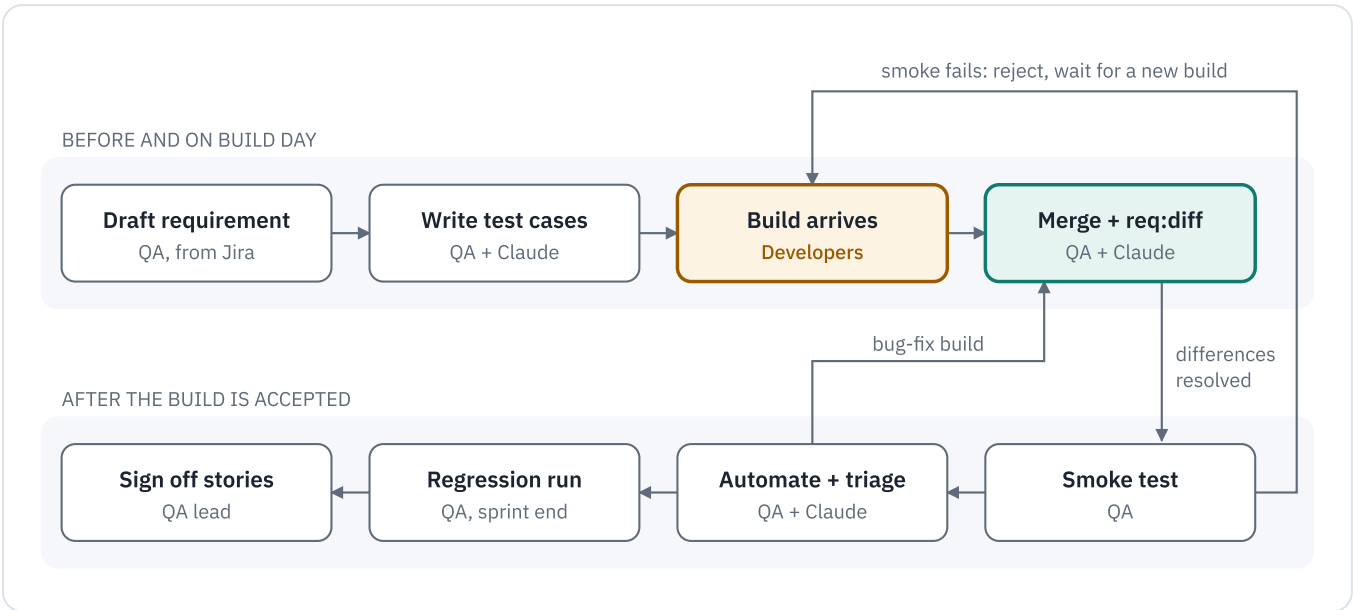
How the QA team tests and automates each sprint. QA writes its own version of every story before the build arrives. On build day it compares that with what the developers actually delivered, then automates against the live build.

WHO USES THE FRAMEWORK QA team only	DEVELOPERS SEND Story MDs, build details, credentials
SOURCE OF TRUTH PER STORY <code>requirements/<platform>/<JIRA>-<module>.md</code>	HOW DIFFERENCES ARE FOUND <code>npm run req:diff</code> after the merge

AT A GLANCE

The sprint at a glance

Developers hand over the build once. QA runs every other step, and two loops send work back when the build isn't ready.



☐ QA step ☒ Developer hand-over ☒ Where QA's expectation meets the build

A two-week sprint, as an example

DAYS 1-2

Planning

Sprint file, one requirement MD per story, from Jira.

DAYS 2-7

Before the build

Test cases reviewed, data and mocks ready, questions sent.

BUILD DAY

Build arrives

Save, credentials, merge, diff, smoke.

FOLLOWING DAYS

Testing

Automate, triage, raise bugs, bug-fix builds.

LAST DAY

Sprint end

Regression, coverage, sign-off.

Day numbers are an example. Your sprint length and build date set the real ones.

Who does what

The framework belongs to QA. Developers never edit it; they only send information with each build.

QA team

Owns the framework

- Writes a requirement MD per story from Jira
- Reviews test cases and generated code
- Merges developer notes and compares them
- Accepts or rejects builds
- Raises bugs, signs off stories

Developers

Send, don't edit

- One MD per story, in any format
- Build details: version, URL, app links
- Credentials through a secure channel
- Answer QA's questions about differences

Claude in VS Code

Does the heavy lifting

- Drafts test cases from requirements
- Merges developer notes into the template
- Finds real locators in the live build
- Writes and runs tests, triages failures

Everything Claude produces is reviewed by QA

Claude drafts; QA decides. A wrong expected result caught at review never becomes a wrong test.

Starting a new web app

Do this once, before the first sprint. It takes about half a day, most of it automating the login. The examples use the app name `prj`.

WHO	NEED FROM DEVELOPERS	RESULT
QA <code>Claude</code>	QA URL, test-ID attribute, one account per role	App folder ready, every role logs in

1

Create the app folder

```
npm run new:app -- prj --platforms web,api
```

Creates `apps/prj/` with `requirements/`, `test-cases/`, `sprints/`, the code folders (`models/`, `pages/`, `data/`, `tests/`), a starter `app.config.ts`, `fixtures.ts` and `.env`. Leave out `api` if you won't write API tests.

2

Make it the active app

In the **root** `.env` of the framework:

```
APP=prj
```

3

Fill in `apps/prj/app.config.ts`

```
web: {
  baseUrl: 'https://qa.prj.example.com', // the QA URL from the
  developers
  testIdAttribute: 'data-testid', // ask them: data-
  testid, data-test or data-qa
},
auth: {
  defaultRole: 'user',
  roles: {
    user: { usernameEnv: 'USER_USERNAME', passwordEnv:
'USER_PASSWORD' },
    admin: { usernameEnv: 'ADMIN_USERNAME', passwordEnv:
'ADMIN_PASSWORD' },
  }
}
```

4

Fill in `apps/prj/.env`

The accounts the developers shared through the password manager, using the variable names from `auth.roles`. If the app sends email OTPs, also fill in the `MAIL_*` settings and run `npm run mail:check`.

5

Automate the login first

Every test starts logged in, so the login comes before any other story.

```
1. write apps/prj/requirements/web/PRJ-001-login.md (from
   templates/requirement.md)
2. /qa-testcases apps/prj/requirements/web/PRJ-001-login.md →
   review the cases
3. /qa-automate apps/prj/test-cases/web/login.testcases.md
4. ask Claude: "wire the login page into auth.login in
   apps/prj/app.config.ts"
5. npm run auth # every role
   must log in
```

6

Put the framework in Git and tell the developers what to send

Commit the new app. Send the developers `templates/dev-handover.md` so they know what to share with every build.

First sprint tips

- The login tests are your first `@smoke` tests. Tag the main happy path of each new story `@smoke` as you automate it, so the smoke suite grows every sprint.
- On a machine with `NODE_ENV=production`, install with `npm install --include=dev`, or the Playwright packages are left out.
- While working on one spec: `npx playwright test <file> --project=web-chrome`.

Sprint planning

Turn each Jira story into a requirement file in the framework's format, before any code exists.

WHO	INPUT	OUTPUT
QA	Jira stories for the sprint, designs if any	Sprint file + one requirement MD per story

1 Create the sprint file

Copy the template and fill in dates, goal, platforms and the story table.

```
copy templates/sprint.md → apps/<app>/sprints/sprint-03.md
```

The story table is your checklist for the sprint: Jira key, platform, requirement file, test-case file, stage, automated count.

2 Write one requirement MD per story and platform

Copy `templates/requirement.md` to `apps/<app>/requirements/web/PRJ-101-login.md` and fill it in from the Jira story:

- Overview, user story and acceptance criteria (Given / When / Then)
- Fields, actions and messages as far as the story describes them
- Business rules, roles, test data needed
- Test ID column: write `TBD`. Nobody knows the locators yet, and that's fine.

A story for both web and mobile gets two files: `requirements/web/...` and `requirements/mobile/...`.

3 Note what's unclear

Missing error texts, unclear rules, unknown roles: list them in the sprint file so you can ask the developers early.

Tip: let Claude draft the requirement

Paste the Jira story into Claude and ask: "Create `apps/prj/requirements/web/PRJ-101-login.md` from this story using `templates/requirement.md`. Leave test IDs as `TBD` and list open questions."

Then review it.

PHASE 2

Before the build

Have reviewed test cases and test data ready, so automation can start the day the build lands.

WHO	INPUT	OUTPUT
QA Claude	Requirement MDs from phase 1	Reviewed test cases, cases baseline, data and mocks

1 Generate test cases

```
/qa-testcases apps/prj/requirements/web/PRJ-101-login.md
```

Claude writes `apps/prj/test-cases/web/login.testcases.md` : one row per case with ID (`TC-LOGIN-01` ...), type, priority, tags, steps, expected result and whether to automate. It covers every acceptance criterion, field rule, boundary, role and UI state.

2 Review every row

This is the most important human step. Fix expected results, remove noise, add what's missing, and answer Claude's open questions. Edit the file directly or ask Claude to change it.

3 The **cases** baseline is recorded

`/qa-testcases` saves a snapshot of the requirement file in `requirements/.baseline/`. This snapshot is **QA's expectation**; on build day, `req:diff` compares against it.

4 Prepare data and states

- Test data created through the API, with cleanup registered
- Error and empty states with `act.mockApi(...)` , so they don't depend on the build
- Per-environment values with `forEnv(...)`

Why draft before the build?

Testing starts on build day instead of days later, and the comparison on build day shows exactly where the developers built something different from the story.

PHASE 3

Build day

Record what arrived, compare it with what QA expected, and decide whether the build is testable. The example build arrives on 15 Oct.

WHO	INPUT	OUTPUT
Developers then QA Claude	Developer MDs, build details, credentials	Merged requirements, decisions on differences, smoke result

1 Save the developer MDs exactly as received

```
apps/prj/sprints/sprint-03/from-dev/2026-10-15/PRJ-101-login.md
apps/prj/sprints/sprint-03/from-dev/2026-10-15/PRJ-102-
dashboard.md
```

Never edit these copies. They are the record of what was delivered, useful when a disagreement comes up later. Add the build to the "Builds received" table in the sprint file.

2 Store the credentials and check the logins

Put accounts only in `apps/prj/.env`, never in any MD file. New role? Add it to `auth.roles` in `app.config.ts` with its `.env` variable names.

```
npm run auth          # logs in every role and saves the sessions
```

3 Merge each developer MD into its requirement file

Developers write in their own format; the requirement file keeps the template format. Ask Claude:

PROMPT FOR CLAUDE

```
Merge the developer notes in apps/prj/sprints/sprint-03/from-
dev/2026-10-15/PRJ-101-login.md
into apps/prj/requirements/web/PRJ-101-login.md. Keep the template
```

structure, fill in the build details, test IDs and exact messages they give, and add a Change log row "Merged dev notes, build 1.4.0". List every point where the developer notes contradict what was there.

4

Compare QA's expectation with the build

```
npm run req:status          # every merged story shows
CHANGED
npm run req:diff -- apps/prj/requirements/web/PRJ-101-login.md

Changes in web/PRJ-101-login.md since QA baseline "Test cases
written"
### Fields
  ~ CHANGED Password: Error message: "Password is required" →
  "Password must be 8+ characters"
  + ADDED Remember me (Type: checkbox; Required: no; Test ID:
  login-remember)
```

Decide on each difference:

Accept

The change is intended (agreed with the PO or developer). Run `/qa-update` on the requirement file. It updates only the affected test cases and tests.

Question

Not sure it's intended. Ask the developer, record the question in the sprint file's "Differences found" table, and wait before changing tests.

Raise a bug

The build contradicts the story. Raise it in Jira. Tests keep the expected behaviour; the failing test gets `knownBug('PRJ-...')`.

5

Smoke test: accept or reject the build

```
npm run test:smoke
```

Green: the build is accepted, testing starts. Red on a core flow (login, main navigation): reject the build, note it in the sprint file, and wait for a new one.

Never put credentials in MD files

Requirement files, test cases and the developer copies are shared and versioned. Only `.env` holds secrets, and it is never committed.

PHASE 4

Testing days

Automate every reviewed test case against the accepted build, and sort every failure into "our problem" or "the app's problem".

WHO	INPUT	OUTPUT
QA Claude	Accepted build, reviewed test cases	Green tests, bugs in Jira, automated baseline

1 Automate each test-case file

```
/qa-automate apps/prj/test-cases/web/login.testcases.md
```

Claude:

- opens the page in the browser, logs in with the right role, and reads the **real** test IDs and error texts from the build
- writes the model (the only place locators live), page, data and spec, reusing what exists
- runs the type check and the spec, and fixes automation problems until green
- keeps assertions when the app behaves differently, and reports a suspected bug instead
- records the `automated` baseline

2 Review the generated code

Like a colleague's pull request: locators only in models, one test per test-case ID, titles `<ID> | <Title>`, no sleeps, no weakened checks, test data cleaned up.

3 Triage failures

```
/qa-fix apps/prj/tests/web/login      # or a tag: /qa-fix @smoke
```

CLAUDE CLASSIFIES IT AS	WHAT HAPPENS
Automation issue	Fixed in the model, page or data
Framework issue	Fixed in <code>src/</code> , only if every app benefits
Application bug	Raised in Jira; assertion kept; <code>knownBug('PRJ-...', '...')</code> as the test's first line
Unclear requirement	A question for you, then the developer

A test that passes only on retry is flaky

It shows up in `reports/flaky.html` . Fix the cause with `npm run flaky:check -- <spec> -g <ID>` ; never add sleeps or retries.

PHASE 5

Bug-fix builds

Every new build repeats the build-day routine, so changes never slip in unnoticed.

1

Save the new developer MDs in a new date folder

```
apps/prj/sprints/sprint-03/from-dev/2026-10-17/PRJ-101-login.md
```

2

Merge, then compare

Same prompt as build day, with the new folder and build number. `req:diff` now shows only what changed since the last time QA processed the file.

3

Update and re-test

```
/qa-update apps/prj/requirements/web/PRJ-101-login.md
```

New cases get the next free IDs, changed cases keep their IDs, removed behaviour is retired with `Automate: no` . IDs are never renumbered. Claude reruns the affected tests

and gives you a re-test list. Remove `knownBug` from tests whose bug is now fixed.

PHASE 6

Sprint end

Prove the whole product still works, and record which version of each story QA signed off.

☐ **Regression suite is green**

`npm run test:regression`, including stories from earlier sprints. Every remaining failure is a ticketed `knownBug`.

☐ **Every test case is automated**

`npm run coverage:tc` shows no MISSING for `Automate: yes` cases.

☐ **No story is left CHANGED or NEW**

`npm run req:status` is clean.

☐ **Each story is signed off**

`npm run req:baseline -- <md> --stage signed-off --by "<name>"`

☐ **Report shared**

`npm run report`; Allure report attached to the Jira stories.

☐ **Sprint file completed**

Builds, differences, bugs and the sign-off line filled in.

0 of 6 done

What developers send with every build

Send the developers `templates/dev-handover.md` once at the start of the project.
With each build they share these three things.

1 One MD file per story

Any format. Named `<JIRA>-<short-name>.md`, e.g. `PRJ-101-login.md`.

- ✓ Jira key, story title, platform
- ✓ Screen location and allowed roles
- ✓ Buttons: when enabled, success, failure
- ✓ Business rules and APIs used
- ✓ Optional: `data-testid` / accessibility IDs
- ✓ Acceptance criteria as implemented
- ✓ Fields: type, required, rules, default
- ✓ Every message, with exact text
- ✓ What changed since the last build

EXAMPLE DEVELOPER NOTE (THEIR OWN FORMAT IS FINE)

```
# PRJ-101 Login (web)
Built: email + password login, "Remember me" checkbox (new).
Errors: empty email -> "Email is required"; wrong password ->
"Invalid email or password".
Password rule changed: min 8 characters -> "Password must be 8+
characters".
Locked after 5 failed attempts -> "Account locked. Try again in 15
minutes."
Test ids: login-email, login-password, login-remember, login-
submit, login-error
Changed since last build: remember-me added, password rule.
```

2 Build details

In the message, or a `build.md` next to the story MDs.

ITEM	EXAMPLE
Build date / version	2026-10-15 / 1.4.0
Platform(s)	web, android
QA URL and app builds	QA URL, APK link, TestFlight link
Stories included	PRJ-101 done · PRJ-103 partial (export not built)
Bug fixes included	PRJ-188, PRJ-190
Known issues	PRJ-103 export button hidden
Flags, DB, config	New role "supervisor"; flag <code>newDashboard</code> on

3 Credentials

Through the password manager or another secure channel. Never inside an MD file, chat message or email body.

Ask for

- ✓ One test account per role
- ✓ Accounts in special states (locked, unverified, expired)
- ✓ OTP approach: fixed QA code, bypass, or test inbox
- ✓ Captcha off on QA, or a test key
- ✓ API keys for test data setup and cleanup

Never accept

- Real customer or production accounts
- Passwords pasted into story MDs
- One shared account for every role
- Personal developer accounts

WHERE QA PUTS THEM: APPS/PRJ/.ENV (NOT COMMITTED)

```
USER_USERNAME=qa.user@prj.test
USER_PASSWORD=.....
ADMIN_USERNAME=qa.admin@prj.test
ADMIN_PASSWORD=.....
```

REFERENCE

Where files live

Requirement and test-case files are organised by platform and module, never by sprint. Everything about a sprint goes in `sprints/`.

<code>apps/prj/</code>	
<code>requirements/</code>	One file per story per platform, owned by QA
<code>web/PRJ-101-login.md</code>	Template format, developer notes merged in
<code>mobile/PRJ-140-login.md</code>	Same story on mobile, its own file
<code>.baseline/</code>	Snapshots QA processed; written by the commands only
<code>test-cases/</code>	QA's reviewed test cases
<code>web/login.testcases.md</code>	From <code>/qa-testcases</code>
<code>sprints/</code>	Sprint bookkeeping only
<code>sprint-03.md</code>	Sprint tracker (<code>templates/sprint.md</code>)
<code>sprint-03/from-dev/2026-10-15/</code>	Developer MDs exactly as received
<code>sprint-03/from-dev/2026-10-17/</code>	Next build's MDs
<code>models/ pages/ data/ tests/</code>	The automation code
<code>.env</code>	Credentials; never committed

QA's MD vs the developers' MD

MD FILE	WRITTEN BY	WHERE IT GOES	EDITED LATER?
QA's MD, from the Jira story	QA	<code>requirements/web/<JIRA>-<module>.md</code>	Yes. Developer notes are merged into it, and the tests are built from it.
Developers' MD, sent with a build	Developers	<code>sprints/sprint-NN/from-dev/<build date>/<JIRA>-<module>.md</code>	No. Kept exactly as received.
Test cases	QA Claude	<code>test-cases/web/<module>.testcases.md</code>	Yes

Use the same Jira key in both file names

PRJ-101-login.md in requirements/web/ and PRJ-101-login.md in from-dev/2026-10-15/ : each developer file clearly belongs to one QA file. A mobile version of the story goes in requirements/mobile/ .

Why requirements aren't sorted by sprint

The change history (baselines) is keyed by the file's path. A story that changes again two sprints later keeps the same file, so req:status and req:diff still work. Moving the file would lose its history.

Keep the framework in Git

Commit requirements/.baseline/ , test-cases/ and sprints/ . Without that, the history req:diff compares against can be lost.

REFERENCE

Worked example: one story through a sprint

PRJ-101 Login (web) in sprint 03, 5 to 16 Oct 2026, with a build on 15 Oct and a bug-fix build on 16 Oct.

PRJ-101 · Login web · sprint 03 · requirement requirements/web/PRJ-101-login.md

DATE	WHAT HAPPENED	FILE / COMMAND
5 Oct	QA writes the requirement from Jira. Test IDs TBD .	requirements/web/PRJ-101-login.md
6 Oct	12 test cases generated and reviewed (TC-LOGIN-01...12).	/qa-testcases
7–14 Oct	Test data and locked-account mock prepared. Question sent: lock after how many attempts?	sprint file

DATE	WHAT HAPPENED	FILE / COMMAND
15 Oct	Build 1.4.0 arrives. Developer note saved, merged.	<code>from-dev/2026-10-15/</code>
15 Oct	Diff: password message changed (accepted), "Remember me" added (accepted), lock text differs from the story (bug PRJ-201).	<code>req:diff</code> → <code>/qa-update</code>
15 Oct	Smoke green. 13 cases automated; TC-LOGIN-09 fails on PRJ-201.	<code>/qa-automate</code> , <code>knownBug</code>
16 Oct	Bug-fix build 1.4.1 fixes PRJ-201. Diff shows the corrected text; <code>knownBug</code> removed, all green.	<code>from-dev/2026-10-16/</code>
16 Oct	Regression green, coverage 13/13, story signed off.	<code>req:baseline --stage signed-off</code>

REFERENCE

Common situations

What to do when the sprint doesn't go to plan.

The developers changed a story mid-sprint

Treat it like a build: save their new MD in a dated folder, merge, `req:diff` , then `/qa-update` .
Never rewrite the requirement from scratch; the diff only works on the same file.

A story arrives only partly built

Keep all its test cases. Mark the ones that can't run yet as `Automate: later` in the test-case file and note it in the sprint file. Switch them back when the rest arrives.

The developers didn't give test IDs

That's fine. `/qa-automate` reads the page and picks the best available locator: role and name, label, placeholder or text. Ask for `data-testid` on elements that turn out unstable.

The build failed smoke

Reject it in the sprint file ("Builds received": rejected) with the failing test IDs and the Allure report. Don't merge or automate against it. Repeat build day when the new build arrives.

The mobile app comes in a later sprint

Add `requirements/mobile/<JIRA>-<module>.md` in that sprint. The web file and its history are not touched.

A story is removed or a behaviour is dropped

Don't delete test cases or renumber IDs. `/qa-update` retires removed cases with `Automate: no`, so reports and history stay consistent.

A new role or login method appears

Add the role to `auth.roles` in `app.config.ts` and its variables to `.env`, then `npm run auth`. Email OTPs: use the test inbox (`npm run mail:check`) or a fixed QA-only code in `OTP_CODE`.

REFERENCE

Commands used in a sprint

In order of use. Slash commands run in the Claude panel in VS Code; `npm run` commands in the terminal.

COMMAND	PHASE	WHAT IT DOES
<code>/qa-testcases <requirement.md></code>	2	Test cases from a requirement file; records the test cases baseline

COMMAND	PHASE	WHAT IT DOES
<code>npm run auth</code>	3	Logs in every role from <code>app.config.</code>
<code>npm run req:status</code>	3, 6	Every story: stage, automated count, CHANGED / NEW flags
<code>npm run req:diff -- <md></code>	3, 5	What changed since QA last processed the file
<code>/qa-update <md></code>	3, 5	Updates only the affected test cases and tests
<code>npm run test:smoke</code>	3	Smoke suite: accept or reject a build
<code>/qa-automate <testcases.md></code>	4	Model, page, data and spec runs until green
<code>/qa-fix [spec @tag]</code>	4	Classifies and fixes failures
<code>npm run test:regression</code>	6	Full regression all sprints so far
<code>npm run coverage:tc</code>	6	Test-case IDs not automated yet

COMMAND	PHASE	WHAT IT DOES
<code>npm run req:baseline -- <md> --stage signed-off --by "<name>"</code>	6	Records QA sign-off of a story
REFERENCE <code>npm run report</code> Glossary	Any	Builds and opens the All report

Requirement file	QA's MD for one story on one platform, in <code>templates/requirement.md</code> format. The single source of truth for the tests.
Developer MD	The developers' own notes for a story, sent with a build. Saved unchanged under <code>sprints/.../from-dev/</code> and merged into the requirement file.
Baseline	A snapshot of a requirement file at a stage QA finished: <code>cases</code> , <code>automated</code> or <code>signed-off</code> .
<code>req:diff</code>	Shows, field by field, how a requirement file differs from its last baseline.
Test-case ID	<code>TC-<MODULE>-NN</code> . One automated test per ID, titled <code><ID> <Title></code> . Never renumbered.
Smoke	The small <code>@smoke</code> suite that decides whether a build is testable.
Regression	The full <code>@regression</code> suite across all sprints.
knownBug	Marks a test that fails because of a ticketed app bug, without weakening its assertion.

New to the team? [QA Team Setup](#). Full details: *Getting Started Guide*, chapter 6 "Working in sprints" (`docs/Getting-Started-Guide.pdf`).

Templates: `templates/sprint.md` · `templates/dev-handover.md` · `templates/requirement.md`

